

JETSON ORIN NANO SUPER

임베디드 Vision VLA 개발 가이드북

하드웨어 · 핀 · JetPack/BSP · 카메라 · AI 가속 · ROS 2 · 안전한 행동 배포

기준: JetPack 7.2.1 / Jetson Linux 39.2.1 / ROS 2 Jazzy

문서 기준일 2026-08-29

사용 안내

이 책은 공식 NVIDIA/ROS 문서를 한국어로 재서술한 실무 해설서다. 전기 절대 정격, 플래시 명령, 보안 키 작업은 설치 릴리스와 동일한 공식 원문을 최종 기준으로 확인한다.

권장 읽기 경로

- 처음 설치: 1장 → 2장 → 3장 → 5장 → 6장
- 커스텀 캐리어보드: 2장 → 4장 → 9장
- Vision VLA: 5장 → 6장 → 7장 → 8장 → 9장

목차

- 1 1. 하드웨어와 전원
- 2 2. 핀과 커넥터
- 3 3. JetPack과 BSP 설치
- 4 4. 커스텀 BSP와 캐리어보드 브링업
- 5 5. 카메라와 Vision 파이프라인
- 6 6. AI 가속과 최적화
- 7 7. ROS 2와 Isaac ROS
- 8 8. 임베디드 Vision VLA 목표와 로드맵
- 9 9. 운영, 안전, 문제 해결
- 10 공식 문서 목록과 한국어 해설 범위
- 11 문서 커버리지 매트릭스

1. 하드웨어와 전원

1.1 보드 정체성

Jetson Orin Nano Super Developer Kit는 새 모듈명이 아니라, Jetson Orin Nano 8GB 개발키트에 높은 전력·클록 프로파일을 제공하는 최신 소프트웨어 구성을 포함한 개발 플랫폼이다. 개발키트 전체 P-number는 P3766, 개발용 microSD 모듈은 P3767-0005, 기준 캐리어보드는 P3768-0000이다.

최신 공식 사용자 가이드는 최대 67 INT8 TOPS, 최대 102 GB/s 메모리 대역폭, 7W~25W 구성 가능 전력을 설명한다. 이 수치는 지원 소프트웨어와 Super 전력 프로파일이 적용된 조건을 전제로 하므로, 실제 애플리케이션은 [nvpmode](#), 열 상태, 메모리 사용량, 입력 해상도에 따라 성능이 달라진다.

1.2 주요 포트

- microSD: 모듈 아래쪽. 개발키트용 루트 파일시스템 저장장치로 사용 가능.
- M.2 Key-M 2280: PCIe 3.0 x4 NVMe용. 모델·데이터셋·컨테이너 때문에 VLA 개발에서는 우선 권장.
- M.2 Key-M 2230: PCIe 3.0 x2 NVMe용.
- M.2 Key-E 2230: 기본 무선 모듈 장착 위치.
- USB-A 4개: USB 3.2 호스트 포트. 각 2단 스택별 VBUS 전류 제한을 고려한다.
- USB-C: 데이터·복구·USB device mode용이며 개발키트 전원 입력이 아니다.
- DisplayPort: 개발키트의 기본 디스플레이 출력. HDMI 직접 출력은 없으며 DP-HDMI 어댑터가 필요하다.
- J20/J21: 22핀 0.5 mm 피치 MIPI CSI 카메라 커넥터.
- J12: 40핀 2x20, 2.54 mm 확장 헤더.
- J14: 12핀 버튼·복구·리셋·디버그 UART 헤더.
- J16: 19V DC 배럴잭, 5.5 mm x 2.5 mm.

1.3 안전한 전원 켜기와 끄기

기본 상태에서는 19V 어댑터를 연결하면 자동으로 부팅한다. 정상 종료는 다음 중 하나를 사용한다.

```
sudo shutdown -h now
```

또는 Ubuntu 데스크톱의 종료 메뉴를 사용한다. 종료가 끝난 뒤에만 DC 전원을 제거한다. 갑작스러운 전원 차단은 보드 반도체보다 microSD/NVMe의 파일시스템, 데이터베이스, 모델 캐시, 컨테이너 레이어를 손상시킬 위험이 더 크다.

물리 전원 버튼

J14에서 다음과 같이 배선한다.

```
J14 5번 — [점퍼캡: 계속 연결] — J14 6번
J14 11번 — [순간식 NO 버튼] — J14 12번
```

- 5-6 단락: 자동 전원 켜기를 비활성화한다.
- 11: GND.
- 12: SLEEP/WAKE*.
- 버튼은 누르는 동안만 접점이 닫히는 momentary, normally-open 형식을 사용한다.
- 래칭 스위치로 11-12를 계속 단락하면 비정상 동작이나 강제 종료가 발생할 수 있다.
- 7-8은 시스템 리셋이므로 전원 버튼과 혼동하지 않는다.

운영 중 짧은 버튼 입력은 Linux의 power-key 이벤트로 전달될 수 있지만 데스크톱 설정에 따라 메뉴·절전 등 동작이 달라진다. 확실한 무인 종료는 systemd 서비스나 MCU가 shutdown을 요청하고 완료를 확인한 뒤 주 전원을 차단하도록 설계한다.

1.4 전력 모드와 모니터링

현재 모드 확인:

```
sudo /usr/sbin/nvpmode -q
sudo tegrastats
```

목록에 표시된 모드 ID로 전환:

```
sudo /usr/sbin/nvpmode -m <mode_id>
```

데스크톱 상단 메뉴에서는 MAXN SUPER를 선택할 수 있다. 장시간 최대 부하는 발열·스로틀링·전원 여유를 반드시 측정해야 한다. nvidia-smi는 Jetson의 주 모니터링 도구가 아니며 tegrastats, 온도 sysfs, nvfancontrol을 사용한다.

1.5 로봇 전원 설계 권장안

- 1 배터리/BMS에서 Jetson용 안정화 19V 레일을 분리한다.
- 2 모터·서보 전원과 Jetson 전원 사이에 접지·노이즈·역기전력 대책을 둔다.
- 3 저전압 경고를 MCU 또는 전원관리보드가 먼저 감지한다.
- 4 MCU가 Jetson에 GPIO/UART/네트워크로 종료 요청을 보낸다.
- 5 Jetson이 종료 완료 신호를 돌려주거나 충분한 제한 시간이 지나면 로드 스위치를 끈다.
- 6 강제 차단은 비상 정지 경로로만 유지하고, 제어 출력은 Jetson과 독립된 안전 회로가 차단한다.

2. 핀과 커넥터

2.1 공통 전기 안전

- J12 인터페이스 신호는 캐리어보드 기준 3.3V 레벨이다.
- 전원 핀과 신호 핀을 혼동하지 않는다. 5V를 GPIO에 인가하지 않는다.
- I2C를 제외한 외부 pull-up/pull-down은 공식 사양에 따라 50 kOhm보다 약한 값만 사용한다.
- 핫플러그 지원이 명시되지 않은 장치는 보드 전원을 끈 뒤 연결한다.
- GPIO 최대 전류는 단순히 3.3V라는 사실만으로 결정할 수 없다. LED·릴레이·모터를 직접 구동하지 말고 트랜지스터, 드라이버, 절연 회로를 사용한다.
- Jetson-IO로 핀 기능을 바꾸기 전 현재 device tree와 연결된 하드웨어를 기록한다.

2.2 J12 40핀 확장 헤더 전체 기본 기능

아래 표는 공식 Carrier Board Specification의 기본 기능을 한국어로 정리한 것이다. 별표가 붙은 신호는 active-low이다.

핀	기본 기능	핀	기본 기능
1	3.3V 전원	2	5V 전원
3	I2C1_SDA	4	5V 전원
5	I2C1_SCL	6	GND
7	GPIO09	8	UART1_TXD
9	GND	10	UART1_RXD
11	UART1_RTS*	12	I2S0_SCLK
13	SPI1_SCK	14	GND
15	GPIO12	16	SPI1_CS1*
17	3.3V 전원	18	SPI1_CS0*
19	SPI0_MOSI	20	GND
21	SPI0_MISO	22	SPI1_MISO
23	SPI0_SCK	24	SPI0_CS0*
25	GND	26	SPI0_CS1*
27	I2C0_SDA	28	I2C0_SCL
29	GPIO01	30	GND
31	GPIO11	32	GPIO07
33	GPIO13	34	GND
35	I2S0_FS	36	UART1_CTS*
37	SPI1_MOSI	38	I2S0_DIN
39	GND	40	I2S0_DOUT

J12는 물리 핀 번호와 Linux GPIO 번호가 동일하지 않다. 보드/SoC 포트명을 Linux line 번호로 바꾸어 사용할 때는 현재 JetPack의 device tree와 [gpioinfo](#) 결과를 기준으로 한다.

J12 전기 특성 핵심

- 3.3V 전원 핀 1, 17: 핀당 1A capability로 표기된다.
- 5V 전원 핀 2, 4: 핀당 1A capability로 표기된다. 입력/출력 전원 경로는 전체 캐리어보드 전원 설계를 함께 확인한다.
- I2C1 핀 3, 5: SoC direct open-drain, 2.2 kOhm pull-up, 데이터시트 VOL을 만족하는 최대 drive는 2mA로 표기된다.
- I2C0 핀 27, 28: open-drain/GPIO, 1.5 kOhm pull-up, 최대 2mA로 표기된다.
- 그 밖의 J12 신호 대부분은 TI TXB0108 level translator를 거친다. 출력 driver가 매우 약한 양방향 변환 구조이므로 LED, 긴 케이블, 큰 capacitance, 강한 pull resistor를 직접 부하로 연결하지 않는다.
- 표의 $\pm 20\mu A$ 를 일반 MCU GPIO의 부하 구동 능력으로 오해하지 않는다. 외부 부하는 buffer/driver로 분리한다.

2.3 J14 버튼 및 디버그 헤더

핀	신호	용도
1	PC_LED-	전원/절전 LED 캐소드
2	PC_LED+	전원/절전 LED 애노드
3	UART2_RXD (DEBUG)	Jetson 수신, USB-TTL TX 연결
4	UART2_TXD (DEBUG)	Jetson 송신, USB-TTL RX 연결
5	AC OK	6번과 연결하면 자동 부팅 비활성화
6	Auto Power-on disable	5번과 점퍼 연결
7	GND	디버그/리셋 기준 접지
8	SYS_RESET*	7번과 순간 연결하면 리셋
9	GND	복구 기준 접지
10	FORCE_RECOVERY*	전원 인가 시 9번과 연결하면 RCM
11	GND	전원 버튼 기준 접지
12	SLEEP/WAKE*	11번과 순간 연결하는 전원 버튼

디버그 UART 어댑터는 반드시 3.3V TTL 레벨을 사용한다. RS-232 전압 어댑터를 직접 연결하지 않는다. 일반 설정은 115200 baud, 8 data bits, no parity, 1 stop bit, no flow control이다.

2.4 Force Recovery Mode

전원이 꺼진 상태:

- 1 J14 9-10을 점퍼로 연결한다.
- 2 USB-C 데이터 케이블을 Ubuntu 호스트와 연결한다.
- 3 DC 전원을 연결한다.
- 4 호스트가 NVIDIA APX/RCM 장치를 인식하면 9-10 점퍼를 제거한다.

전원이 켜진 상태에서는 9-10을 연결한 뒤 7-8을 잠깐 단락하여 리셋할 수 있다. 플래시 완료 후 복구 점퍼를 반드시 제거한다.

호스트 확인 예:

```
lsusb
```

2.5 CSI 카메라 커넥터

- J20(CAM0): 2-lane CSI 구성.
- J21(CAM1): 2-lane 또는 4-lane CSI 구성.
- 두 커넥터 모두 22핀, 0.5 mm pitch, bottom-contact FFC 형식이다.
- Raspberry Pi Camera Module v2처럼 15핀 카메라는 15-to-22핀 변환 케이블이 필요하다.
- FFC의 접점 방향을 사진만 보고 추정하지 말고 공식 Hardware Layout과 카메라 모듈 공급자 배선도를 함께 확인한다.
- CSI 카메라는 전원을 제거한 뒤 장착/분리한다.

J20 Camera #0 전체 핀

핀	신호	전기 방향/설명
1	+3.3V	카메라 전원
2	CAM_I2C_SDA	양방향 3.3V, mux 카메라측 1 kOhm pull-up
3	CAM_I2C_SCL	출력 3.3V, mux 카메라측 1 kOhm pull-up
4	GND	접지
5	CAM0_MCLK	출력 1.8V
6	CAM0_PWDN	출력 1.8V
7	GND	접지
8	CSI0_D1_P	CSI 입력
9	CSI0_D1_N	CSI 입력
10	GND	접지
11	CSI0_D0_P	CSI 입력
12	CSI0_D0_N	CSI 입력
13	GND	접지
14	CSI1_CLK_P	CSI clock 입력
15	CSI1_CLK_N	CSI clock 입력
16	GND	접지
17	CSI1_D1_P	CSI 입력
18	CSI1_D1_N	CSI 입력
19	GND	접지
20	CSI1_D0_P	CSI 입력
21	CSI1_D0_N	CSI 입력
22	GND	접지

J21 Camera #1 전체 핀

핀	신호	전기 방향/설명
1	+3.3V	카메라 전원
2	CAM_I2C_SDA	양방향 3.3V, mux 카메라측 1 kOhm pull-up
3	CAM_I2C_SCL	출력 3.3V, mux 카메라측 1 kOhm pull-up
4	GND	접지
5	CAM1_MCLK	출력 1.8V
6	CAM1_PWDN	출력 1.8V
7	GND	접지
8	CSI3_D1_P	CSI 입력
9	CSI3_D1_N	CSI 입력
10	GND	접지
11	CSI3_D0_P	CSI 입력
12	CSI3_D0_N	CSI 입력
13	GND	접지
14	CSI2_CLK_P	CSI clock 입력
15	CSI2_CLK_N	CSI clock 입력
16	GND	접지
17	CSI2_D1_P	CSI 입력
18	CSI2_D1_N	CSI 입력
19	GND	접지
20	CSI2_D0_P	CSI 입력
21	CSI2_D0_N	CSI 입력
22	GND	접지

카메라 모듈 배선은 센서별 리셋, 전원 시퀀스, MCLK 요구사항이 다르므로 일반 22핀 케이블만으로 호환성을 판단할 수 없다. **PWDN**과 **MCLK**는 1.8V 신호임을 특히 주의한다.

2.6 기타 헤더

J17 선택 CAN 헤더

핀	신호	설명
1	CAN_TX	출력 3.3V
2	CAN_RX	입력 3.3V
3	GND	접지
4	3.3V	전원

J17은 CAN controller logic 신호용이며 CANH/CANL 버스가 아니다. 외부 CAN transceiver와 termination이 필요하다. 선택 footprint이므로 실장 여부를 확인한다.

J13 팬 헤더

핀	신호	설명
1	GND	접지
2	5V	팬 전원
3	GPIO08	fan tachometer 입력 5V
4	GPIO14	fan PWM 출력 5V

커넥터는 4핀 1.25 mm pitch 계열이다. 일반 PC 2.54 mm 팬 커넥터를 억지로 끼우지 않는다.

J3 RTC 백업 배터리

핀	신호	설명
1	PMIC_BBAT	RTC backup supply
2	GND	접지

2핀 1.25 mm pitch 커넥터이다. 셀 종류·전압·극성을 공식 supported component와 PMIC 요구사항에서 확인하고 충전식 셀을 임의로 연결하지 않는다.

J16 DC 잭

- 입력 범위: 9V~20V DC.
- barrel: 길이 9.5 mm, 외경 5.5 mm, 내경/핀 2.5 mm.
- center positive.
- 잭 최대 지원 전류: 3.5A.
- 핀 1: DC input, 핀 2/3: GND.

개발키트 기본 어댑터는 19V를 사용한다. 다른 전원을 쓸 때는 단순 전압뿐 아니라 transient, ripple, cable drop, inrush를 검증한다.

J19 PoE와 J18 backpower

- J19 1~4: Ethernet RJ45의 PoE VC1~VC4를 꺼내는 header.
- PoE 38V~60V를 캐리어보드가 요구하는 5V~20V로 낮추는 별도 converter가 필요하다.
- J18 핀 1: main DC input 9V~20V, 최대 3A. 핀 2: GND.
- J19를 곧바로 Jetson 전원 입력으로 연결하면 안 된다.

2.7 Jetson-IO 사용

```
sudo /opt/nvidia/jetson-io/jetson-io.py
```

Jetson-IO에서 40핀 헤더를 선택한 뒤 compatible hardware 또는 manual configuration으로 기능을 지정하고 저장한다. 저장 결과는 `/boot/`의 device-tree overlay(DTBO)로 내보낼 수 있다. 변경 후 재부팅하고 다음을 확인한다.

```
sudo /opt/nvidia/jetson-io/config-by-hardware.py -l
gpioinfo
```

Jetson-IO는 배선의 전압·전류 안전을 보장하지 않는다. 핀mux가 맞더라도 외부 회로가 5V logic이면 레벨 시프터가 필요하다.

3. JetPack과 BSP 설치

3.1 기준 버전

2026-08-29 기준 공식 최신 개발키트 경로는 JetPack 7.2.1, Jetson Linux 39.2.1이다. JetPack에는 BSP와 CUDA, cuDNN, TensorRT, VPI, 멀티미디어 API, 컨테이너 런타임 등 가속 소프트웨어가 포함된다.

현재 상태 확인:

```
cat /etc/nv_tegra_release
dpkg-query -W nvidia-jetpack 2>/dev/null || true
uname -a
```

JetPack 7.2 시스템은 R39 계열을 보고해야 한다. 프로젝트가 JetPack 6.x에 고정되어 있다면 r36.5 문서를 별도 기준선으로 유지하고 r39 명령을 섞지 않는다.

3.2 저장장치 선택

- microSD: 64GB UHS-1 이상 권장. 입문과 복구가 쉽지만 지속적인 컨테이너-데이터 기록에는 한계가 있다.
- NVMe: Vision VLA 개발의 기본 권장. 모델, rosbag, Docker overlay, 빌드 캐시 때문에 256GB 이상, 실무는 512GB 이상이 편하다.
- USB 저장장치: 일부 플래시/부팅 경로에서 지원되지만 로봇의 진동-케이블 신뢰성과 성능을 고려한다. 백업 없이 저장장치 플래시를 실행하지 않는다. `lsblk -o NAME,SIZE,MODEL,SERIAL,MOUNTPOINTS`로 대상 장치를 두 번 확인한다.

3.3 JetPack 7.2.1 권장: Jetson ISO

JetPack 7.2부터 개발키트용 SD Card Image 방식 대신 Jetson ISO 설치 USB를 사용한다. ISO 파일을 microSD에 직접 기록하면 안 된다.

- 1 PC에서 공식 JetPack 7.2.1 ISO를 받는다.
- 2 16GB 이상 USB 메모리에 ISO를 기록한다.
- 3 Jetson의 대상 microSD 또는 NVMe를 장착한다.
- 4 UEFI/QSPI 펌웨어가 36.x 세대 이상인지 확인한다.
- 5 USB 설치 미디어로 부팅해 대상 저장장치에 Jetson Linux를 설치한다.
- 6 초기 설정과 EULA를 완료한다.
- 7 설치 후 `sudo apt update && sudo apt dist-upgrade`를 수행한다.
- 8 필요한 JetPack 구성 요소를 설치한다.

```
sudo apt update
sudo apt install nvidia-jetpack
sudo reboot
```

3.4 구형 출고 펌웨어의 QSPI 업데이트

JetPack 7.2 ISO는 JetPack 6.x 세대 UEFI/QSPI 펌웨어가 필요하다. UEFI 메뉴에서 펌웨어 버전을 확인한다.

- 모니터: DisplayPort와 키보드를 연결하고 NVIDIA 로고 직후 Esc를 반복 입력한다.
- 헤드리스: J14 디버그 UART(3 RX, 4 TX, 7 GND)를 연결하고 부팅 중 Esc를 입력한다.
- 36.0보다 오래된 펌웨어라면 공식 [JetPack 6.x Update Path](#)를 먼저 수행한다.

공식 업데이트 경로의 핵심은 구형 부트 펌웨어가 읽을 수 있는 JetPack 5.1.3 microSD 이미지로 먼저 부팅하고, 패키지 업데이트를 통해 QSPI를 JetPack 6.x 세대로 올린 다음, 최종 JetPack 설치 미디어로 전환하는 것이다. 중간 재부팅과 QSPI 갱신이 끝나기 전에 전원을 차단하지 않는다.

3.5 SDK Manager 경로

SDK Manager는 Ubuntu x86_64 호스트가 필요하다.

- 1 호스트에 SDK Manager를 설치한다.
- 2 Target Hardware에서 Jetson Orin Nano를 선택한다.
- 3 Jetson을 Force Recovery Mode로 전환한다.
- 4 저장장치로 NVMe 또는 SD Card를 선택한다.
- 5 OEM Configuration은 일반적으로 Runtime을 선택한다.
- 6 플래시 완료 후 J14 recovery 점퍼를 제거한다.
- 7 초기 부팅을 완료한 뒤 필요하면 SDK Manager로 추가 JetPack 구성요소를 설치한다.

3.6 명령행 BSP/플래시 경로

명령행 경로는 자동화, 커스텀 rootfs, 양산 플래시, 파티션 변경에 사용한다.

- 1 해당 Jetson Linux 릴리스의 Driver Package와 Sample Root Filesystem을 받는다.
- 2 Ubuntu x86_64 호스트에서 [Linux_for_Tegra](#)를 준비한다.
- 3 sample rootfs를 [Linux_for_Tegra/rootfs](#)에 푼다.
- 4 공식 스크립트로 NVIDIA 바이너리를 적용한다.
- 5 대상 보드를 RCM에 넣는다.
- 6 r39.2 문서의 [l4t_initrd_flash.sh](#)를 사용한다.

Orin NX/Nano의 공식 플래시 방식은 initrd flash이다. [flash.sh](#)는 개발 중 일부 파티션 작업에만 사용하고, OTA를 고려하는 제품 이미지는 initrd 레이아웃을 유지한다.

일반 NVMe 플래시 형태:

```
cd Linux_for_Tegra
sudo ./tools/kernel_flash/l4t_initrd_flash.sh \
--external-device nvme0n1p1 \
-p "-c ./bootloader/generic/cfg/flash_t234_qspi.xml" \
-c ./tools/kernel_flash/flash_l4t_t234_nvme.xml \
--showlogs --network usb0 --erase-all \
jetson-orin-nano-devkit external
```

Super 구성을 명시적으로 사용할 때는 현재 릴리스에 존재하는 [jetson-orin-nano-devkit-super.conf](#)와 Quick Start의 명령을 확인한다. 보드 설정 이름을 임의로 추측하지 않는다.

3.7 설치 검증

```
cat /etc/nv_tegra_release
uname -r
lsblk -o NAME,SIZE,MODEL,MOUNTPOINTS
sudo /usr/sbin/nvpmode1 -q
sudo tegrastats
nvcc --version || true
python3 - <<'PY'
try:
    import tensorrt as trt
    print('TensorRT', trt.__version__)
except Exception as exc:
    print('TensorRT 확인 실패:', exc)
PY
```

검증 결과와 설치 날짜, ISO/Driver Package 체크섬, board SKU, 저장장치 모델을 프로젝트 운영 문서에 남긴다.

4. 커스텀 BSP와 캐리어보드 브링업

4.1 BSP의 범위

Jetson Linux BSP는 부트 펌웨어, UEFI, Linux 커널, Ubuntu rootfs, NVIDIA 드라이버, device tree, 플래시 도구와 저수준 플랫폼 설정을 포함한다. P3768 기준 캐리어보드에서 개발만 한다면 수정 없이 사용할 수 있지만, 생산용 캐리어보드로 옮기면 다음 항목을 보드에 맞춰야 한다.

- kernel device tree
- MB1 BCT(핀mux, GPIO, pad voltage, 전원 관련 부트 설정)
- MB2 설정
- ODM data와 UPHY lane 구성
- EEPROM/board ID 정책
- flash configuration과 partition layout
- USB, PCIe, NVMe, Ethernet, display, camera, fan, 전원 시퀀스

4.2 기준 자료

- 1 Jetson Orin NX Series and Orin Nano Series Design Guide
- 2 Jetson Orin Nano Developer Kit Carrier Board Specification
- 3 Carrier Board Reference Design Files(회로도, PCB, BOM)
- 4 Jetson Orin NX/Nano Pinmux Configuration Template
- 5 Jetson Linux r39.2.1 Developer Guide의 Module Adaptation and Bring-Up
- 6 Jetson Orin Technical Reference Manual(TRM)
- 7 모듈 Data Sheet 및 Thermal Design Guide

하드웨어 설계 능력과 BSP 지원 범위는 다를 수 있다. Design Guide에 전기적으로 가능한 인터페이스가 있어도 현재 Jetson Linux 릴리스가 해당 조합을 지원하는지 Software Feature Overview와 Release Notes에서 확인한다.

4.3 보드 명명과 형상관리

커스텀 보드 이름은 소문자 영숫자, 하이픈, 밑줄을 사용하고 공백을 피한다. 다음을 한 세트로 버전 관리한다.

- 보드 이름과 revision
- SOM SKU/P-number
- carrier board ID와 EEPROM 내용
- pinmux spreadsheet 원본과 생성된 .dtsi
- device tree source/overlay
- MB1/MB2 BCT 변경
- flash .conf 파일
- partition XML
- kernel config 및 out-of-tree 모듈
- rootfs customization 스크립트

- 플래시 로그와 검증 결과
- NVIDIA 원본 파일을 직접 덮어쓰기보다 커스텀 파일을 별도 디렉터리에 두고, 원본 릴리스 태그와 diff를 유지한다.

4.4 권장 브링업 순서

단계 1: 전원 전용 검증

- SOM 미장착 상태에서 전원 레일, 시퀀스, 리플, inrush, 역극성 보호를 검증한다.
- 각 레일의 최대·최소와 ramp timing을 Design Guide와 대조한다.
- DC 입력, power-good, shutdown request, reset 신호를 오실로스코프로 확인한다.

단계 2: 최소 부팅

- 디버그 UART를 먼저 확보한다.
- QSPI/UEFI, RAM, 기본 rootfs가 부팅하는지 확인한다.
- 부트 로그를 원본 P3768 로그와 비교한다.

단계 3: 저속 인터페이스

- EEPROM, I2C, SPI, UART, GPIO, fan tach/PWM을 순차 검증한다.
- 각 기능마다 pinmux와 device tree node가 동시에 맞는지 확인한다.

단계 4: 고속 인터페이스

- USB, PCIe/NVMe, Ethernet, CSI, DisplayPort 순으로 lane mapping과 signal integrity를 확인한다.
- UPHY lane은 한 기능의 변경이 다른 PCIe/USB 포트에 영향을 줄 수 있으므로 전체 lane map을 표로 관리한다.

단계 5: 열·전력·복구

- 모든 전력 모드에서 스트레스 테스트한다.
- 팬 고장, 센서 단절, 저전압, 과열, 강제 복구, 전원 재인가를 시험한다.

4.5 Pinmux와 GPIO

공식 Pinmux spreadsheet에서 보드 설정을 만들고 생성된 pinmux/gpio dtsi를 BSP의 T234 BCT 경로에 반영한다. JetPack 6 이후에는 Jetson-IO 또는 pinmux dtsi 방식이 가능하지만, 생산 보드의 부트 초기 상태와 pad voltage는 MB1 적용 여부를 분명히 해야 한다.

GPIO 식별 절차:

- 1 module pin name을 Design Guide/Pin Function Names Guide에서 확인한다.
 - 2 SoC pad와 GPIO port를 TRM에서 확인한다.
 - 3 pinmux에서 GPIO 기능, 방향, pull, tristate를 설정한다.
 - 4 생성된 dtsi와 최종 DTB에 값이 들어갔는지 검사한다.
 - 5 런타임에서는 character-device GPIO API와 [gpioinfo](#)를 사용한다.
- `/sys/class/gpio` 숫자를 다른 JetPack 버전에서 그대로 복사하지 않는다.

4.6 Device Tree

장치 노드에는 최소한 compatible, reg/address, interrupt, clocks, resets, regulators, pinctrl, status를 정확히 기술한다. overlay를 사용할 경우 base DTB와 overlay의 symbol/fragment 호환성을 확인한다.

검증 예:

```
sudo dtc -I fs -O dts /proc/device-tree > running-device-tree.dts
grep -R "status = \"okay\"" running-device-tree.dts
dmesg -T | grep -Ei 'error|fail|timeout|i2c|spi|pcie|camera'
```

4.7 커널과 드라이버

- r39.2의 정확한 kernel source/tag와 toolchain을 사용한다.
- 기본 커널 ABI와 NVIDIA out-of-tree 모듈의 버전을 맞춘다.
- 카메라 센서 드라이버는 loadable kernel module 패키지로 관리하는 것을 우선 고려한다.
- source, config, DTB, modules, initrd를 동일 빌드 ID로 묶는다.
- 설치 전에 복구 가능한 기존 커널/DTB와 UART 콘솔을 확보한다.

4.8 Rootfs 커스터마이징과 재현성

rootfs에 손으로 패키지를 설치한 뒤 이미지를 복제하는 방식보다, 패키지 목록·APT pin·사용자 생성·systemd 서비스·컨테이너 이미지를 스크립트로 재현한다. [nv_customize_rootfs.sh](#), Debian 패키지, cloud-init 또는 자체 provisioning을 사용하되 비밀번호와 키를 이미지에 하드코딩하지 않는다.

4.9 양산 전 체크리스트

- initrd flash 기반의 양산 이미지와 massflash 절차 검증
- A/B rootfs 및 OTA 전략
- Secure Boot, 키 보관, 디스크 암호화 여부 결정
- 고유 장치 ID, 인증서, SSH host key 생성 정책
- watchdog, crash log, health telemetry
- factory test: RAM, storage, USB, Ethernet, CSI, GPIO, thermal, fan
- 라이선스 고지와 소스 제공 의무 검토
- 전원·EMC·열·기구·케이블 스트레인 릴리프 검증

퓨즈 기반 Secure Boot는 되돌릴 수 없는 단계가 포함될 수 있다. 개발키트 한 대로 즉시 적용하지 말고 키 복구·RMA·양산 서명 체계를 먼저 확정한다.

5. 카메라와 Vision 파이프라인

5.1 입력 방식 선택

입력	일반 경로	장점	주의점
CSI Bayer	libargus / nvarguscamerasrc	Jetson ISP 사용, 낮은 복사 비용	센서 드라이버와 ISP 튜닝 필요
CSI RAW	V4L2 direct	드라이버 검증, RAW 캡처	ISP 후처리 없음
USB UVC	V4L2 / v4l2src	연결과 호환이 비교적 쉬움	USB 대역폭·지연·압축 형식
GMSL/CoE	Argus 또는 SIPL 경로	장거리·다중 카메라	serializer/deserializer, 동기화, 드라이버 통합

r39.2 문서는 Jetson 전체 로드맵에서 SIPL을 주요 신규 카메라 경로로 설명하지만, Orin Nano의 MIPI CSI와 기존 ISP 워크플로에서는 Argus가 여전히 핵심이다. SIPL의 모든 Thor 기능이 Orin Nano에서 동일하다고 가정하지 말고 Camera Support Matrix를 확인한다.

5.2 첫 카메라 검증

장치 확인:

```
v4l2-ctl --list-devices
v4l2-ctl --device=/dev/video0 --list-formats-ext
media-ctl -p
```

CSI Argus 미리보기 예:

```
gst-launch-1.0 nvarguscamerasrc sensor-id=0 ! \
  'video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1' ! \
  nvvidconv ! autovideosink
```

USB UVC 예:

```
gst-launch-1.0 v4l2src device=/dev/video0 ! \
  videoconvert ! autovideosink
```

실제 format은 `v4l2-ctl --list-formats-ext` 결과와 맞춘다. 헤드리스 환경에서는 display sink 대신 encoder/filesink 또는 fakesink를 사용한다.

5.3 카메라 드라이버 브링업

Bayer 센서는 다음을 함께 구현해야 한다.

- 1 I2C 센서 드라이버와 register table
- 2 전원 rail, clock, reset/powerdown 시퀀스
- 3 V4L2 controls와 mode table
- 4 device tree의 module, sensor, endpoint, CSI/VI topology
- 5 lane 수, clock, pixel format, line length, frame length
- 6 Argus 사용 시 ISP tuning 및 camera core 연동

직접 V4L2 경로는 RAW 검증에 적합하다. ISP를 사용하는 제품은 libargus 또는 GStreamer `nvarguscamerasrc` 경로를 검증해야 한다. NVIDIA 공개 BSP에는 모든 센서용 ISP tuning 도구가 포함되지 않으므로 인증 카메라 파트너의 지원 범위를 확인한다.

5.4 Zero-copy에 가까운 파이프라인

Vision VLA의 병목은 모델만이 아니라 프레임 복사와 색공간 변환이다. 가능한 경우 다음 경로를 유지한다.

```
CSI/USB 입력
-> NVMM/NvBufSurface
-> VIC 또는 CUDA 전처리
-> TensorRT/Isaac ROS NITROS 추론
-> 소형 구조화 결과
-> VLM 또는 행동 정책
```

CPU OpenCV `cv::Mat`로 매 프레임 복사한 뒤 다시 GPU로 올리는 구조는 해상도와 FPS가 높을수록 손해가 커진다. GStreamer caps에서 `memory:NVMM`, Isaac ROS에서는 NITROS 협상을 확인한다.

5.5 VPI 사용

VPI는 CPU, CUDA, VIC, OFA 등 가속 백엔드에서 비동기 Vision 연산을 구성한다. Orin Nano는 PVA가 없는 구성일 수 있으므로 VPI 문서의 플랫폼별 backend 표를 확인한다. 전처리, remap, optical flow, stereo, feature detection을 GPU 모델과 병렬화할 때 유용하다.

VPI 설계 원칙:

- payload와 buffer를 매 프레임 생성하지 않고 재사용한다.
- stream과 event로 단계 사이를 동기화한다.
- backend 변경 전 정확도·latency·memory copy를 함께 측정한다.
- 입력 해상도별 payload 제약을 확인한다.

5.6 데이터셋 수집 규칙

- 카메라 원본 시간, ROS timestamp, robot state, action, instruction을 같은 episode ID로 묶는다.
- 자동 노출/화이트밸런스와 수동 고정 조건을 모두 기록한다.
- calibration 파일, lens/focus, 해상도, FPS, sensor mode를 메타데이터에 남긴다.
- 실패 행동과 안전 개입도 삭제하지 않고 별도 레이블로 보존한다.
- 사람·문서·주거 공간 촬영 시 개인정보와 동의 절차를 둔다.

5.7 캘리브레이션과 시간 동기화

VLA가 행동을 출력하려면 이미지와 관절 상태의 시간 정렬이 중요하다. 카메라 intrinsics, distortion, camera-to-base extrinsics를 버전 관리하고, 다중 카메라는 hardware trigger/PTP 가능 여부를 검토한다. 소프트웨어 timestamp만 사용할 때는 capture time과 message publish time을 구분해 latency budget에 포함한다.

6. AI 가속과 최적화

6.1 설치 전략

JetPack 호스트 패키지와 컨테이너를 무작정 혼합하지 않는다.

- 호스트 설치: 카메라·GStreamer·ROS 장치 접근이 단순하고 디버깅이 쉽다.
- 컨테이너: Python/CUDA 프레임워크 버전을 고정하고 재현하기 쉽다.
- 권장: BSP·드라이버·장치 서비스는 호스트, 모델 런타임은 JetPack 호환 컨테이너로 분리한다.

컨테이너 기본 준비:

```
sudo apt update
sudo apt install -y nvidia-container curl jq
curl https://get.docker.com | sh
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl daemon-reload
sudo systemctl restart docker
sudo usermod -aG docker "$USER"
```

새 로그인 후 `docker info`로 NVIDIA runtime을 확인한다. 인터넷에서 받은 Docker 스크립트를 실행하는 정책은 조직 보안 기준에 맞게 바꾼다.

6.2 PyTorch와 TensorRT 역할

- PyTorch: 모델 실험, 전처리 검증, export, 일부 추론.
- ONNX: 프레임워크와 배포 런타임 사이의 교환 형식.
- TensorRT builder: 해당 GPU와 shape/precision에 맞춘 engine 생성.
- TensorRT runtime: 생성된 engine을 낮은 지연으로 실행.

Jetson에서는 JetPack과 호환되는 NVIDIA PyTorch wheel 또는 NGC 컨테이너를 사용한다. 일반 PyPI 패키지가 Jetson의 CUDA/cuDNN 조합과 맞는다고 가정하지 않는다.

6.3 TensorRT 기본 루프

```
PyTorch checkpoint
-> ONNX export
-> ONNX checker / shape 확인
-> TensorRT engine build
-> 정확도 비교
-> latency-throughput-memory 측정
-> 애플리케이션 통합
```

설치된 `trtexec` 옵션을 먼저 확인한다.

```
trtexec --help
```

전형적인 예시는 다음과 같지만, TensorRT 버전과 모델의 dynamic input 이름에 맞게 수정한다.

```
trtexec --onnx=model.onnx \
--saveEngine=model_fp16.engine \
--fp16 \
--memPoolSize=workspace:1024 \
--profilingVerbosity=detailed
```

TensorRT engine은 GPU 아키텍처·TensorRT 버전·builder 설정에 종속될 수 있다. PC에서 만든 engine을 Jetson에 무조건 복사하지 말고 대상 Jetson에서 빌드하거나 공식 호환 조건을 확인한다.

6.4 정밀도 전략

- FP32: 기준 정확도와 디버깅.
- FP16: Orin Nano의 일반적인 첫 최적화 선택.
- INT8: calibration/quantization 품질 검증이 필요하지만 메모리와 처리량에 유리.
- INT4/AWQ 등: VLM 메모리 절감에 유용하지만 모델-런타임별 지원과 품질 저하를 측정한다.

정확도 검증은 전체 평균 하나가 아니라 작은 물체, 어두운 장면, 반사, motion blur, 언어 지시의 동의어, 안전 관련 클래스별로 수행한다.

6.5 8GB 메모리 예산

통합 메모리를 CPU와 GPU가 공유하므로 다음 항목을 동시에 계산한다.

- OS와 데스크톱
- 카메라 buffer/NVMM
- ROS 2 queue와 rosbag
- TensorRT workspace/KV cache
- VLM weights와 activation
- 행동 정책
- Docker daemon과 overlay cache

권장 운영 순서:

- 1 GUI가 필요 없으면 headless 운영을 검토한다.
- 2 VLM은 약 2B~4B급 또는 강하게 양자화된 모델부터 시작한다.
- 3 Vision detector/segmenter는 낮은 해상도와 필요한 FPS로 제한한다.
- 4 카메라 frame queue를 무제한으로 두지 않는다.
- 5 swap은 OOM 완화용이지 실시간 latency 해결책이 아니다.
- 6 모델을 동시에 상주시킬 필요가 없으면 단계별 로드/언로드를 고려한다.

6.6 측정 항목

```
sudo tegrastats --interval 1000
```

기록할 값:

- end-to-end sensor-to-action latency와 p95/p99
- 카메라 FPS와 drop 수
- GPU/CPU/EMC 사용률
- RAM/SWAP
- 온도, throttle, 전력 모드
- TensorRT build 시간과 engine 크기
- 정확도와 안전 실패율

모델 단독 FPS만으로 로봇 성능을 판단하지 않는다. 제어 주기, 메시지 복사, VLM 호출 주기, actuator 응답까지 포함한다.

6.7 권장 프로세스 분리

```
camera_service    : 캡처·timestamp·재연결  
perception_service : detection/depth/pose  
vlm_service       : 저주기 의미 추론  
policy_service    : 상태+목표 -> 후보 행동  
safety_supervisor : 제한·충돌·watchdog·E-stop  
actuator_bridge   : 검증된 명령만 MCU/모터에 전달  
recorder          : episode/telemetry/실패 로그
```

VLM 프로세스가 죽거나 늦어져도 actuator가 마지막 명령을 계속 유지하지 않도록 timeout을 둔다.

7. ROS 2와 Isaac ROS

7.1 기준 스택

JetPack 7.2.1은 Ubuntu 24.04 계열과 ROS 2 Jazzy 조합을 기준으로 잡는다. 최신 Isaac ROS 문서는 Jetson Orin과 Jetson Thor에서 JetPack 7.2, ROS 2 Jazzy, 128GB 이상 NVMe를 지원 기준으로 제시한다. 실제 Orin Nano 8GB에서는 패키지별 메모리 요구량을 확인하고 필요한 구성만 설치한다.

ROS 2 설치에 ROS 공식 Ubuntu 24.04/Jazzy 절차를 따른다. 저장소 키와 패키지 설치 명령은 시간이 지나면 바뀔 수 있으므로 가이드북의 복사본보다 공식 설치 페이지를 우선한다.

설치 확인:

```
source /opt/ros/jazzy/setup.bash
ros2 doctor --report
ros2 run demo_nodes_cpp talker
```

다른 터미널:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

7.2 Isaac ROS 개발 환경

최신 Isaac ROS는 `isaac-ros-cli` 관리 환경을 권장한다. Docker 격리는 재현성이 높고, virtual environment는 호스트 장치 접근이 단순하며, bare metal은 고급 사용자용이다.

기본 준비:

```
sudo apt-get update
sudo apt-get install -y git-lfs python3-colcon-common-extensions python3-rosdep
git lfs install --skip-repo
```

Docker 모드의 개념적 순서:

- 1 Jetson용 Docker/NVIDIA runtime을 정상화한다.
- 2 `isaac-ros-cli`를 설치한다.
- 3 `sudo isaac-ros init docker`로 환경을 초기화한다.
- 4 센서 패키지와 프로젝트 의존성을 계층형 설정으로 추가한다.
- 5 `quickstart rosbag`으로 먼저 검증한 뒤 실제 카메라로 바꾼다.

Isaac ROS는 특정 CUDA, TensorRT, OpenCV 버전을 고정할 수 있다. JetPack 기본 OpenCV와 충돌할 때 시스템 패키지를 즉시 제거하기 전에 Docker 격리를 우선 사용한다.

7.3 NITROS

NITROS는 ROS 2 노드 사이의 GPU 친화적 형식 협상과 type adaptation을 통해 불필요한 CPU 복사를 줄인다. 카메라에서 DNN까지 NITROS가 유지되는지 로그와 negotiated format을 확인한다. 중간에 일반 `sensor_msgs/Image` 변환이나 Python 노드가 끼면 host copy가 다시 생길 수 있다.

7.4 권장 Vision 구성 요소

- image pipeline: rectify, resize, format conversion
- TensorRT inference: detector, segmenter, keypoint, depth
- visual SLAM: 위치와 자세 추정

- nvblox: depth 기반 3D 재구성/장애물 지도
 - AprilTag: 기준 마커와 캘리브레이션
 - FoundationPose/pose estimation: 물체 자세
 - cuMotion/MoveIt 연동: 조작 경로 생성
- 모든 패키지를 한 번에 설치하지 말고 VLA 목표에 필요한 최소 그래프부터 구성한다.

7.5 QoS와 실시간성

- 카메라: SensorDataQoS, 작은 depth, best-effort를 우선 검토한다.
- 상태/명령: 신뢰성이 중요하지만 오래된 명령을 쌓지 않도록 queue depth와 deadline을 설계한다.
- 정적 calibration: transient-local을 사용할 수 있다.
- 제어 명령: timestamp, sequence, expiry를 포함한다.
- watchdog: deadline miss, process heartbeat, actuator acknowledgement를 감시한다.

7.6 TF와 좌표계

최소 프레임:

```
map -> odom -> base_link -> camera_link -> camera_optical_frame
                        -> arm_base -> tool0
```

이미지 좌표, optical frame, robot base, tool frame을 혼용하면 올바른 인식도 잘못된 행동으로 변환된다. URDF와 calibration 결과를 단일 저장소에서 관리하고, TF tree를 매 배포마다 검사한다.

```
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo base_link camera_optical_frame
```

7.7 시뮬레이션과 HIL

- SIL: x86_64에서 Isaac Sim과 ROS 2 애플리케이션을 함께 시험한다.
- HIL: Isaac Sim은 x86_64에서 실행하고 Jetson은 실제 대상 하드웨어에서 perception/policy를 실행한다.
- 실제 로봇 전에는 recorded rosbag replay로 deterministic regression을 만든다.

Isaac Sim을 Orin Nano에서 직접 실행하는 구성은 목표로 삼지 않는다. 시뮬레이션은 고성능 x86 GPU에서, 배포는 Jetson에서 수행하는 역할 분리가 현실적이다.

8. 임베디드 Vision VLA 목표와 로드맵

8.1 목표 정의

최종 목표는 카메라 영상과 자연어 지시, 로봇 상태를 입력으로 받아 검증된 행동을 출력하는 임베디드 시스템이다. Orin Nano 8GB에서는 단일 거대 end-to-end VLA보다 계층형 구조가 안전하고 구현 가능성이 높다.

카메라/상태
 -> 실시간 Vision 인식
 -> 저주기 VLM 의미 추론-계획
 -> 제한된 action schema
 -> 정책/궤적 생성
 -> 독립 safety supervisor
 -> MCU/actuator

8.2 성공 기준

- 기능: 정해진 5~10개 작업을 언어 지시로 수행한다.
- 지연: 제어 루프는 고정 주기, VLM은 비동기 저주기로 분리한다.
- 안전: 사람/충돌/통신 손실/모델 timeout에서 정지한다.
- 재현성: 동일 software manifest와 calibration으로 다시 설치할 수 있다.
- 관측성: episode마다 영상, 상태, 지시, 후보 행동, 거부 이유를 기록한다.
- 배포성: 부팅 후 자동 시작, watchdog, OTA rollback이 가능하다.

8.3 단계별 마일스톤

M0. 보드 기준선

- JetPack 7.2.1 설치와 NVMe 부팅
- MAXN SUPER/25W 모드 비교
- [tegrastats](#) 기록
- 안전 종료 버튼과 원격 SSH
- 결과: 2시간 연속 부하에서 오류·열 스로틀링 상태를 설명할 수 있음

M1. 카메라 기준선

- CSI 또는 USB 카메라 1대
- 고정 해상도/FPS, timestamp, calibration
- GStreamer/NVMM 파이프라인
- 결과: 30분 캡처에서 frame drop과 latency 측정

M2. 실시간 perception

- detector/segmenter/depth 중 작업에 필요한 1~2개만 TensorRT로 배포
- ROS 2 topic과 TF 연결
- 결과: sensor-to-result p95 latency와 작업별 정확도 확보

M3. 언어 기반 목표 해석

- 소형 VLM/LLM이 자연어를 자유 텍스트 대신 제한된 JSON action schema로 변환
- 허용 동작, 좌표, 속도, confidence, expiry 포함
- 결과: 잘못된 JSON, 미지원 지시, 불확실한 지시를 거부

M4. 행동 정책

- 먼저 규칙/상태기계 또는 작은 policy network로 action primitive 실행
- pick, place, move, stop 같은 primitive를 ROS action으로 구현
- 결과: VLM이 직접 PWM/속도 명령을 만들지 않음

M5. imitation/VLA

- teleoperation episode 수집
- x86 GPU에서 behavior cloning 또는 경량 VLA fine-tuning
- Jetson용 양자화·TensorRT/지원 런타임 변환
- 결과: 고정 평가 세트에서 성공률, 개입률, 안전 거부율 측정

M6. 제품화

- systemd/container auto-start
- read-only 또는 A/B rootfs 검토
- OTA, 로그 회수, crash recovery
- 전원·열·진동·네트워크 단절 시험

8.4 모델 크기 전략

NVIDIA의 Jetson 자료는 Orin Nano 8GB에서 대략 4B 이하의 효율적 LLM/VLM부터 시작하도록 안내한다. 실제 VLA는 vision encoder, language model, action head와 ROS 프로세스가 메모리를 함께 쓰므로 더 작은 모델이 필요할 수 있다.

- VLM: 2B~4B급 4-bit 후보를 벤치마크한다.
- perception: TensorRT FP16, 필요 시 INT8.
- action policy: 별도 작은 네트워크 또는 state machine.
- 긴 영상은 모든 프레임을 VLM에 넣지 않고 detector/scene-change로 keyframe을 선택한다.
- KV cache와 이미지 token 수를 제한한다.

8.5 Action schema 예시

```
{
  "intent": "pick",
  "target": "red_block",
  "frame": "base_link",
  "constraints": {
    "max_speed_mps": 0.10,
    "keep_upright": true
  },
  "confidence": 0.86,
  "expires_ms": 1000
}
```


이 결과는 명령이 아니라 후보다. safety supervisor가 대상 존재, 좌표 유효성, workspace, 속도, collision, freshness를 검사한 뒤 승인한다.

8.6 안전 아키텍처

- 물리 E-stop은 Jetson 소프트웨어를 거치지 않고 actuator power/enable을 차단한다.
- Jetson heartbeat가 사라지면 MCU가 모터를 정지한다.
- 모델 출력에는 허용 목록과 단위·범위 검증을 적용한다.
- 자유 텍스트를 shell, ROS service 이름, CAN frame으로 직접 실행하지 않는다.
- 카메라가 가려지거나 TF가 오래되면 행동을 중지한다.
- VLM confidence만 안전 근거로 사용하지 않는다.

8.7 평가 데이터셋

각 작업에 대해 정상, 조명 변화, 배경 변화, 가림, 유사 물체, 모호한 지시, 위험 지시, 네트워크 단절, camera freeze를 포함한다. 보고 지표:

- task success rate
- intervention rate
- false action / unsafe proposal rate
- perception miss/false positive
- instruction rejection accuracy
- sensor-to-action p50/p95/p99
- thermal steady-state 성능

8.8 현실적인 첫 프로젝트

고정 카메라와 2~4축 소형 로봇을 사용해 "색 블록을 찾아 지정 구역으로 옮기기"를 구현한다. perception은 TensorRT detector, 언어는 5개 intent schema, 행동은 사전 검증된 primitive, 안전은 workspace와 속도 제한으로 구성한다. 이 프로젝트가 안정화된 뒤 end-to-end imitation policy를 추가한다.

9. 운영, 안전, 문제 해결

9.1 부팅 전 체크

- 19V 정격 어댑터와 DC 잭 규격 확인
- CSI/J12/J14 배선은 전원 제거 후 점검
- fan connector와 방열판 체결 확인
- microSD/NVMe 장착 확인
- Force Recovery 점퍼 제거 확인
- 모터 전원은 비활성 상태에서 첫 부팅

9.2 기본 진단 묶음

```
date
cat /etc/nv_tegra_release
uname -a
cat /etc/os-release
lsblk -o NAME,SIZE,MODEL,SERIAL,FSTYPE,MOUNTPOINTS
df -hT
free -h
sudo /usr/sbin/nvpmode1 -q
systemctl --failed
journalctl -b -p warning..alert --no-pager
dmesg -T | grep -Ei 'error|fail|timeout|thrott|overcurrent|camera|nvme|usb'
```

로그를 공유할 때 비밀번호, 토큰, Wi-Fi 설정, 인증서를 제거한다.

9.3 부팅하지 않을 때

- 1 전원 LED와 어댑터 출력 확인.
- 2 J14 5-6, 7-8, 9-10 점퍼 위치 확인.
- 3 모든 USB/GPIO/카메라를 제거하고 최소 구성으로 부팅.
- 4 J14 UART에서 가장 이른 부트 로그 확보.
- 5 UEFI에서 저장장치가 보이는지 확인.
- 6 QSPI와 rootfs 버전 호환성 확인.
- 7 필요하면 RCM과 SDK Manager/ISO 복구 경로 사용.

9.4 카메라가 안 보일 때

- FFC 접점 방향과 CAM0/CAM1 포트 확인
- sensor driver/device tree overlay 설치 여부
- `dmesg`, `media-ctl -p`, `v4l2-ctl --list-devices`
- `nvargus-daemon` 상태와 로그
- 지원 sensor mode와 GStreamer caps 일치
- I2C ACK, MCLK, reset, regulator 순서를 하드웨어에서 측정

`sudo systemctl restart nvargus-daemon`은 일시 복구에 도움이 될 수 있으나, 반복 장애의 원인은 driver, cable, power, caps, buffer leak에서 찾아야 한다.

9.5 CUDA/TensorRT/Docker 문제

- container tag가 JetPack/Jetson aarch64와 호환되는지 확인
- [docker info](#)에서 NVIDIA runtime 확인
- 컨테이너 내부에서 GPU 접근 테스트
- TensorRT engine을 만든 버전과 실행 버전 비교
- OOM이면 model weights, KV cache, camera buffer, ROS queue를 따로 측정
- `tegrastats`로 thermal throttle과 EMC 병목 확인

9.6 저장장치 보호

- 중요한 데이터는 NVMe에 기록하고 정기 백업한다.
- `rosbag/log`에 크기·보존 기간 제한을 둔다.
- disk full 전에 경고한다.
- 비정상 전원 차단이 예상되면 journaling, fsync 정책, read-only rootfs, 별도 data partition을 검토한다.
- 업데이트 전 GPT/partition layout과 boot firmware 호환성을 확인한다.

9.7 systemd 운영 원칙

- 네트워크, 카메라, actuator 준비 순서를 dependency로 표현한다.
- `Restart=on-failure`, `StartLimit*`, `watchdog`를 사용한다.
- 서비스가 죽을 때 actuator를 안전 상태로 보내는 종료 핸들러를 둔다.
- stdout 무제한 기록을 피하고 `journald` rate/size 제한을 둔다.
- root 권한이 필요하지 않은 AI 서비스는 일반 사용자로 실행한다.

9.8 보안 최소 기준

- 기본 계정·비밀번호 제거, SSH key 사용
- 불필요한 서비스와 포트 비활성화
- 컨테이너에 `--privileged`를 기본으로 주지 않음
- 카메라와 actuator 장치만 필요한 group/device 권한 부여
- 모델과 업데이트 이미지에 checksum/signature 적용
- 생산 전 Secure Boot, disk encryption, OTA rollback 설계
- SBOM과 오픈소스 라이선스 목록 유지

9.9 릴리스 체크리스트

- [] JetPack/L4T/커널/컨테이너 tag 고정
- [] 공식 release notes의 known issues 검토
- [] 모든 핀과 케이블 도면 revision 일치

- [] 2시간 이상 열 평형 부하 시험
- [] 전원 차단·저전압·재부팅 시험
- [] camera disconnect/reconnect 시험
- [] 네트워크 단절과 VLM timeout 시험
- [] E-stop과 MCU watchdog 시험
- [] 복구 이미지와 UART 접근 확보
- [] 데이터·모델·로그 백업/삭제 정책 확인

공식 문서 목록과 한국어 해설 범위

기준일: 2026-08-29. 링크의 latest 문서는 이후 변경될 수 있으므로, 제품 기준선에서는 열람일과 버전을 함께 기록한다.

A. 개발키트와 설치

- 1 [Jetson Orin Nano Developer Kit User Guide](#)
- 반영: 제품 정체성, 성능, 구성, 설치 문서 구조.
- 1 [Quick Start - JetPack 7.2.1](#)
- 반영: Jetson ISO, 대상 storage, firmware gate, 첫 부팅.
- 1 [BSP Setup](#)
- 반영: ISO, SDK Manager, flash script, RCM.
- 1 [JetPack 6.x Update Path](#)
- 반영: 구형 QSPI/UEFI 업데이트 필요성.
- 1 [Docker Setup](#)
- 반영: NVIDIA container runtime, Docker 검증.
- 1 [CUDA Setup](#)
- 반영: host/컨테이너 CUDA 선택.
- 1 [JetPack SDK Setup](#)
- 반영: nvidia-jetpack 패키지, 버전 확인.
- 1 [JetPack SDK Downloads and Notes](#)
- 기준선: JetPack 7.2.1, L4T 39.2.1, CUDA 13.2.1, TensorRT 10.16.2.

B. 하드웨어와 핀

- 1 [Hardware Layout](#)
- 반영: 포트 위치·역할, DP, USB-C, NVMe, CSI.
- 1 [How-to Guides](#)
- 반영: 전원 종료, power mode, tegrastats, CSI 연결, RCM.
- 1 [Carrier Board Specification PDF](#)
- 반영: J12 40핀 전체 기본 기능, J14 전체, 카메라·팬·CAN·RTC·전원 주의사항.
- 1 [Jetson Orin NX/Nano Design Guide](#)
- 반영: 커스텀 캐리어보드 설계 검토 체계.
- 1 [Jetson Download Center](#)
- 검색 대상: Pinmux template, Data Sheet, Thermal Design Guide, reference design files, supported components.

C. Jetson Linux BSP

- 1 [Jetson Linux r39.2 Developer Guide](#)
- 반영: BSP 전체 구조와 버전 기준선.
- 1 [r39.2.1 Quick Start](#)
- 반영: P-number, board configuration, Super 설정.
- 1 [Flashing Support](#)
- 반영: initrd flash, NVMe/SD/USB, flash.sh 제한, A/B-암호화 확장.
- 1 [Module Adaptation and Bring-Up](#)

- 반영: hardware/software bring-up checklist.
- 1 [Orin NX and Nano Adaptation](#)
- 반영: P3767/P3768, MB1/MB2, DT, GPIO, USB, PCIe, UPHY.
- 1 [Configuring Expansion Headers](#)
- 반영: Jetson-IO와 DTBO.
- 1 [Platform Power and Performance](#)
- 반영: nvpmoel, fan, thermal, clocks, 장기 MAXN 주의.
- 1 [Secure Boot for Jetson Orin](#)
- 반영: 제품화 체크리스트와 키/복구 주의.

D. 카메라·멀티미디어·AI

- 1 [Camera Development](#)
- 반영: Argus/SIPL 방향성과 support matrix 확인.
- 1 [Camera Software Development Solution](#)
- 반영: libargus, nvarguscamerasrc, V4L2 API matrix.
- 1 [Sensor Software Driver Programming](#)
- 반영: sensor driver, device tree, mode table, controls.
- 1 [Jetson Linux Multimedia](#)
- 반영: NVMM, accelerated GStreamer/FFmpeg 설계 방향.
- 1 [VPI 4.1 Getting Started](#)
- 반영: stream, buffer, backend, 비동기 Vision 파이프라인.
- 1 [TensorRT Documentation](#)
- 반영: ONNX-engine-runtime, precision, profiling, engine 호환성.
- 1 [PyTorch for Jetson](#)
- 반영: JetPack 호환 wheel/container 사용 원칙.

E. ROS 2, Isaac ROS, VLM/VLA

- 1 [ROS 2 Jazzy Documentation](#)
- 반영: Ubuntu 24.04/Jazzy, QoS, colcon/rosdep 기본 원칙.
- 1 [Isaac ROS Getting Started](#)
- 반영: JetPack 7.2, ROS 2 Jazzy, NVMe, isaac-ros-cli, SIL/HIL.
- 1 [Isaac ROS DNN Inference](#)
- 반영: TensorRT/Triton ROS 노드와 NITROS.
- 1 [Jetson Edge AI: LLMs, VLMs, Robotics](#)
- 반영: Orin Nano 8GB의 현실적 소형 VLM 범위.
- 1 [Jetson Orin Nano Super Boost](#)
- 반영: Vision Transformer/VLM 활용 방향.

문서 사용 규칙

- 하드웨어 절대 정격과 pin electrical limits: Carrier Board Specification/Data Sheet 원문 우선.
- BSP 명령: 설치된 L4T와 동일한 archive 버전 문서 우선.
- latest AI 문서: JetPack에 실제 포함된 TensorRT/VPI 버전과 support matrix 우선.

- 블로그: 아키텍처와 모델 후보 참고용이며 제품 요구사항의 최종 근거로 사용하지 않는다.

문서 커버리지 매트릭스

이 표는 “무엇이 빠졌는지”를 추적하기 위한 인덱스다. 완료는 한국어 실무 설명이 있다는 뜻이며 공식 원문 표 전체를 복제했다는 뜻이 아니다.

영역	공식 근거	한국어 위치	상태	원문 직접 확인이 필요한 항목
제품/P-number	User Guide, r39.2 Quick Start	01, 03	완료	주문 SKU/PCN
포트와 저장장치	Hardware Layout	01	완료	기구 envelope
정상 종료/전원 버튼	How-to, Carrier Spec J14	01, 02	완료	switch debounce/timing 상세
J12 40핀 기본 핀	Carrier Spec Fig 3-1/Table 3-3	02	완료	drive current, SoC pin, default pull
J14 12핀	Carrier Spec Table 3-4	02	완료	LED 전기 상세
CSI 포트	Hardware Layout, Carrier Spec	02, 05	완료	J20/J21 개별 전원-lane 핀 표
팬/CAN/RTC/PoE	Carrier Spec	02	요약 완료	connector part, 전압/전류, 실장 옵션
Jetson ISO	Quick Start 7.2.1	03	완료	PC별 ISO 기록 UI
구형 QSPI 업데이트	Update Path	03	완료	각 화면 캡처와 패키지 버전
SDK Manager	BSP Setup	03	완료	GUI 버전별 화면
initrd flash	Flashing Support	03, 04	완료	모든 workflow 옵션/A-B/암호화 조합
Super config	Quick Start, release notes	03	완료	릴리스별 정확한 conf 목록
custom carrier	Adaptation Guide	04	완료	실제 고객 회로별 DT/BCT 값
pinmux/GPIO	Pinmux template, TRM	04	완료	spreadsheet 각 셀과 GPIO 계산
USB/PCIe/UPHY	Design/Adaptation Guide	04	절차 완료	보드별 lane map/SI 계산
Secure Boot/OTA	Security, Flashing	04, 09	정책 완료	실제 키-푸즈-양산 명령
Argus/V4L2	Camera docs	05	완료	sensor별 register와 tuning blob
SIPL	Camera Development r39.2	05	방향 완료	플랫폼별 current support matrix
GStreamer/NVMM	Multimedia Guide	05	완료	모든 plugin property
VPI	VPI 4.1	05	완료	알고리즘별 backend 제한
Docker/CUDA	Devkit setup	06	완료	선택한 container tag compatibility
PyTorch/TensorRT	NVIDIA DL docs	06	완료	모델별 export/quantization recipe
ROS 2 Jazzy	ROS docs	07	완료	네트워크별 DDS tuning
Isaac ROS/NITROS	Isaac ROS docs	07	완료	선택 package quickstart 전체
Vision VLA 목표	NVIDIA Jetson/Isaac 자료	08	완료	특정 로봇-데이터셋-모델 선정
safety supervisor	시스템 설계 지침	08, 09	완료	인증/법규/위험 분석
진단/운영	User/BSP docs	09	완료	현장별 로그 수집 backend

완료 정의

- 1 관련 공식 문서가 목록에 있다.

- 2 한국어 가이드의 대응 장이 있다.
- 3 위험한 명령과 전기 작업에 경고가 있다.
- 4 버전 종속 내용은 기존 버전을 적었다.
- 5 고객 보드·센서·로봇에 따라 결정할 값은 원문 직접 확인 항목으로 남겼다.